

Programowanie obiektowe

Wykład 7
Kompozycja

Kompozycja

- **Kompozycja** to technika tworzenia złożonych obiektów poprzez łączenie ich z innymi, mniejszych obiektów, które pełnią rolę części składowych.
- Kompozycja to relacja typu **część-całość** (*whole-part*) lub **"ma"** (*has-a*). Oznacza, że jeden obiekt (całość, kompozyt) składa się z innych obiektów (części, komponentów).
- Jeden obiekt **"ma"** inny obiekt jako swoje pole (atrybut) i wykorzystuje jego funkcjonalność,.
- Przykłady:
 - Samochód ma Silnik .
 - Faktura składa się z Pozycji .
 - Książka ma Rozdziały .
 - Klub ma Zawodników .
 - Projekt zawiera Etapy .

Kompozycja w kodzie

- Realizacja przez pole (atrybut) w klasie – kompozycja najczęściej wyrażana jest przez zawieranie instancji jednej klasy jako pola innej.

```
class Silnik
{
    public void Uruchom()
    {
        Console.WriteLine("Silnik został uruchomiony.");
    }
}

class Samochod
{
    private Silnik silnik;

    public Samochod()
    {
        silnik = new Silnik(); // Kompozycja – samochód zawiera silnik
    }

    public void Jedz()
    {
        silnik.Uruchom();
        Console.WriteLine("Samochód jedzie.");
    }
}
```

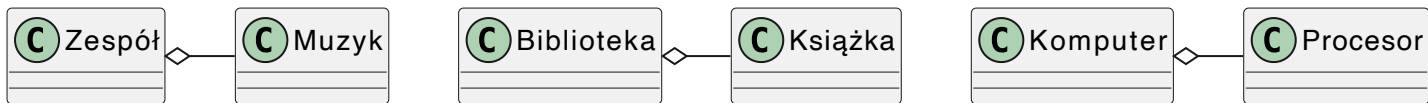
Kompozycja a agregacja

- **Kompozycja** to silna relacja typu „część-całość” między obiektami.
- Jeden obiekt (całość) **składa się** z innych obiektów (części), które **nie mogą istnieć samodzielnie** poza tą całością.
- Całość **zarządza cyklem życia** swoich części – gdy zostaje usunięta, jej części również przestają istnieć.
- Kompozycja wskazuje na bardzo ścisłe powiązanie między obiektami, gdzie jeden obiekt jest **integralnym elementem** drugiego.
- Przykład: pomieszczenie jest częścią mieszkania – bez mieszkania nie ma sensu mówić o pomieszczeniu jako oddzielnym bycie.
- W UML oznaczane symbolem **wypełnionego rombu** po stronie **całości**.



Kompozycja a agregacja

- **Agregacja** to słabsza relacja typu „ma” (has-a), również wskazująca, że jeden obiekt **zawiera** inny, ale w sposób **luźniejszy**.
- Obiekty części mogą istnieć niezależnie od całości i mogą być **współdzielone** przez wiele obiektów nadrzędnych.
- Agregacja pokazuje, że istnieje **związek logiczny**, ale nie fizyczna zależność między obiektami – całość nie odpowiada za istnienie części.
- Przykład: zespół muzyczny składa się z muzyków, ale ci muzycy mogą grać też w innych zespołach – nie są wyłączną częścią jednego zespołu.
- W UML oznaczane symbolem **pustego rombu** po stronie **całości**.



W praktyce to często decyzja projektowa, nie techniczna, a granica między kompozycją a agregacją bywa umowna.

Kompozycja z obiektami automatycznymi

- Kompozycja może być realizowana przez obiekty automatyczne (np. w C++) - tworzone jako pola klasy.
- Ten sposób jest bezpieczny pamięciowo i naturalny dla silnych relacji, gdzie składnik jest integralną częścią całości.

```
class Silnik {
public:
    void uruchom() {
        std::cout << "Silnik uruchomiony" << std::endl;
    }
};

class Samochod {
    Silnik silnik; // obiekt automatyczny – kompozycja

public:
    void jedz() {
        silnik.uruchom();
        std::cout << "Samochód jedzie" << std::endl;
    }
};

int main() {
    Samochod auto1; // silnik tworzony automatycznie razem z samochodem
    auto1.jedz();
    return 0;
}
```

Tworzenie i usuwanie komponentów automatycznych

- W C++ konstrukcja i destrukcja komponentów odbywa się automatycznie.
 - **Najpierw** wywoływane są **konstruktory wszystkich pól składowych** (komponentów), w kolejności ich deklaracji w klasie.
 - **Potem** wykonywane jest ciało **konstruktora klasy nadrzędnej**.
 - W momencie usuwania obiektu, **najpierw** wykonywane jest ciało **destruktora klasy nadrzędnej**.
 - **Potem** wywoływane są **destruktory pól składowych**, w odwrotnej kolejności do konstruktorów.

```
class Silnik {  
public:  
    Silnik() { cout << "Najpierw wykonuje się konstruktor Silnika\n"; }  
    virtual ~Silnik() { cout << "Na samym końcu: destruktory Silnika\n"; }  
};  
  
class Samochod {  
    Silnik silnik;  
  
public:  
    Samochod() { cout << "Jako drugi wykona się konstruktor Samochodu\n"; }  
    virtual ~Samochod() { cout << "Przy usuwaniu: najpierw destruktory Samochodu\n"; }  
};
```

Wywoływanie konstruktorów komponentów automatycznych

- Jeśli konstruktor komponentu przyjmuje parametr, w klasie całości należy użyć listy inicjalizacyjnej w konstruktorze.

```
class Silnik {  
    int pojemnosc;  
  
public:  
    Silnik(int p) : pojemnosc(p) { }  
};  
  
class Samochod {  
    Silnik silnik; // kompozycja  
  
public:  
    // Konstruktor samochodu przyjmuje pojemność i przekazuje ją do silnika  
    Samochod(int pojemnosc) : silnik(pojemnosc) { }  
};
```


Przekazywanie komponentów do konstruktora

- Przekazywanie **gotowego obiektu komponentu** jako parametru konstruktora klasy całości:
 - daje większą kontrolę nad konfiguracją kompozycji.

```
class Samochod {  
    Silnik silnik; // kompozycja  
  
public:  
    // Konstruktor przyjmuje obiekt Silnik i kopiuje/przenosi go do pola  
    Samochod(const Silnik& s) : silnik(s) { }  
};
```

- W powyższym przykładzie wywoła się **konstruktor kopiujący** klasy `Silnik`.
- C++11 umożliwia **przeniesienie własności** – pozwala to na przekazanie zasobów do nowego obiektu, zamiast ich kopiowania.

```
Samochod(Silnik&& s) : silnik(std::move(s)) { }
```

- Wymaga to konstruktora przenoszącego w klasie `Silnik`

```
Silnik(Silnik&& other) noexcept : pojemnosc(other.pojemnosc) {  
    // ...  
}
```

Kompozycja z obiektami dynamicznymi

- W C++ możemy realizować kompozycję z obiektami tworzonymi na stercie (i używając wskaźników).

```
class Samochod {
    Silnik* silnik;

public:
    Samochod(int pojemnosc) {
        silnik = new Silnik(pojemnosc); // dynamiczne tworzenie komponentu
        cout << "Konstruktor Samochod" << endl;
    }

    virtual ~Samochod() {
        delete silnik; // obowiązkowe zwolnienie pamięci
        cout << "Destruktor Samochod" << endl;
    }

    void jedz() const {
        silnik->uruchom();
        cout << "Samochód jedzie" << endl;
    }
};
```

- Obiekt "całość" zarządza czasem życia obiektu "części" - odpowiada za jego stworzenie i usunięcie.

Kompozycja z typami referencyjnymi

- W Javie i C# tak wygląda kompozycja z typami referencyjnymi.

```
public class Samochod {  
    private Silnik silnik;  
  
    public Samochod() {  
        this.silnik = new Silnik(1600); // silnik tworzony wewnątrz  
    }  
  
    public void jedz() {  
        silnik.uruchom();  
        System.out.println("Samochód jedzie.");  
    }  
}
```

- Obiekt "całości" przechowuje jako pole **referencję do** obiektu "części".
- W tym przykładzie obiekt komponentu jest tworzony wewnątrz klasy (*silna kompozycja*).

Przekazywanie referencji na komponent

- Czas życia komponentu nie musi być ściśle powiązany z cyklem życia całości.

```
public class Samochod {  
    private Silnik silnik;  
  
    public Samochod(Silnik silnik) {  
        this.silnik = silnik; // kompozycja z przekazany obiekt  
    }  
}
```

- Obiekt komponentu może być przekazany z zewnątrz (np. przez konstruktor).

```
// Kod klienta  
Silnik s = new Silnik(2000);  
Samochod auto = new Samochod(s);  
auto.jedz();
```

- To bardziej elastyczne podejście.
- *Nie stanowi to problemu w językach zarządzanych, ale w C++ trudne może być rozstrzygnięcie kto odpowiada za usunięcie komponentu, dlatego lepiej korzystać z przenoszenia własności lub wskaźników inteligentnych (`unique_ptr`, `shared_ptr`).*

Współdzielone komponenty

- Kompozycja przez typy referencyjne lub wskaźniki pozwala, by wiele obiektów korzystało z tego samego komponentu.

```
// Jeden silnik
Silnik s = new Silnik(2000);

// Dwa samochody
Samochod auto1 = new Samochod(s);
Samochod auto2 = new Samochod(s);
```

- Niekiedy jest to błędem projektowym (każdy **Samochód** powinien mieć swój własny **Silnik**).
 - Gdy komponent powinien mieć tylko jednego właściciela.
 - Gdy stan komponentu zależy od kontekstu (całości), np.
- A niekiedy normalną, dopuszczalną praktyką (wiele elementów systemu korzystających z tego samego **Loggера** , wiele klas współdzielących połączenie z bazą danych).
 - Gdy komponent jest kosztowny w tworzeniu, a współdzielenie pozwala na oszczędzenie zasobów.
 - Gdy komponent jest bezstanowy (*stateless*) lub niezmienny (*immutable*).
 - Gdy uzasadnia to logika biznesowa.
- W tej sytuacji własność jest współdzielone, a cykl życia zarządzany z zewnątrz.

Kopiowanie komponentów

- Jeśli z jednej strony, chcemy przekazywać komponent jako parametr konstruktora, a jednocześnie chcemy uniknąć współdzielenia, powinniśmy kopiować komponent w konstruktorze.

```
public class Samochod {  
    private Silnik silnik;  
  
    public Samochod(Silnik silnikZewnetrzny) {  
        this.silnik = new Silnik(silnikZewnetrzny); // kopiowanie przez konstruktor  
    }  
}
```

- To wymaga istnienia konstruktora kopiującego w klasie `Silnik`.

```
public class Silnik {  
    private int pojemnosc;  
  
    public Silnik(int pojemnosc) {  
        this.pojemnosc = pojemnosc;  
    }  
  
    public Silnik(Silnik inny) {  
        this.pojemnosc = inny.pojemnosc;  
    }  
}
```

Hermetyzacja w kompozycji

- Kompozycja nie narusza zasady hermetyzacji.
- Obiekt całości posługuje się komponentem przez publiczny interfejs.

```
public class Zamowienie {  
    private Adres adresDostawy;  
    private double wartoscZamowienia;  
  
    public Zamowienie(Adres adres, double wartosc) {  
        this.adresDostawy = adres;  
        this.wartoscZamowienia = wartosc;  
    }  
  
    public double obliczKosztDostawy() {  
        int odleglosc = adresDostawy.getOdlegloscOdMagazynu();  
        double koszt = 10.0 + odleglosc * 0.5; // stała opłata + 0.5 zł/km  
        return koszt;  
    }  
}
```

- Obiekt całości do realizacji swych zadań wykorzystuje metody komponentu:
 - Komponent może pełnić rolę danych lub narzędzia pomocniczego.
 - Obiekt całości wykonuje swoją logikę, wspomagając się komponentem.

Delegowanie zadań do komponentu

- Obiekt całości nie zawsze sam realizuje logikę – może ją **delegować do komponentu**.
- Komponent staje się odpowiedzialny za konkretne zachowanie.
- Obiekt całości pełni rolę koordynatora lub interfejsu.

```
public class PolitykaRabatu {  
    public double obliczRabat(double kwota) {  
        if (kwota > 1000) return 0.1;  
        if (kwota > 500) return 0.05;  
        return 0;  
    }  
}  
  
public class Zamowienie {  
    private double kwota;  
    private PolitykaRabatu polityka;  
  
    public Zamowienie(PolitykaRabatu polityka) {  
        this.polityka = polityka;  
    }  
  
    public double obliczRabat() {  
        return polityka.obliczRabat(kwota); // delegacja do komponentu  
    }  
}
```


Agregacja z wieloma komponentami

- Poniższy przykład pokazuje agregację z dowolną liczbą komponentów: obiekt całości nie zarządza cyklem życia swych komponentów, a komponenty mogą być współdzielone.

```
public class Druzyna {  
    private String nazwa;  
    private Zawodnik[] zawodnicy;  
    private int liczbaZawodnikow;  
  
    public Druzyna(String nazwa, int maxZawodnikow) {  
        this.nazwa = nazwa;  
        this.zawodnicy = new Zawodnik[maxZawodnikow];  
        this.liczbaZawodnikow = 0;  
    }  
  
    public void dodajZawodnika(Zawodnik z) {  
        // (warto sprawdzić, czy nie przekraczamy dozwolonego rozmiaru)  
        zawodnicy[liczbaZawodnikow++] = z;  
    }  
  
    public void wyswietlSkład() {  
        System.out.println("Drużyna: " + nazwa);  
        for (int i = 0; i < liczbaZawodnikow; i++) {  
            System.out.println("- " + zawodnicy[i].getImie());  
        }  
    }  
}
```

Kompozycja z wieloma komponentami

- Obecność tablicy i zmieniającej się w czasie życia liczby komponentów nie oznacza automatycznie luźnego związku: w tym przykładzie obiekt całości sam zarządza swoimi komponentami, które nie istnieją poza nim.

```
public class Faktura {
    private String numer;
    private PozycjaFaktury[] pozycje;
    private int liczbaPozycji;

    public Faktura(String numer, int maksPozycji) {
        this.numer = numer;
        this.pozycje = new PozycjaFaktury[maksPozycji];
        this.liczbaPozycji = 0;
    }

    public void dodajPozycje(String nazwa, int ilosc, double cena) {
        if (liczbaPozycji < pozycje.length) {
            pozycje[liczbaPozycji++] = new PozycjaFaktury(nazwa, ilosc, cena);
        } else {
            System.out.println("Za dużo pozycji na fakturze!");
        }
    }

    public void wyswietl() { ... }
}
```

Korzystanie z metod komponentów

- Komponent udostępnia logikę, którą obiekt całości wykorzystuje do realizacji zwich zadań.

```
public class PozycjaFaktury {
    private String nazwaProduktu;
    private int ilosc;
    private double cenaJednostkowa;
    ...
    public double obliczWartosc() {
        return ilosc * cenaJednostkowa;
    }
}

public class Faktura {
    private String numer;
    private PozycjaFaktury[] pozycje;
    private int liczbaPozycji;
    ...
    public double obliczSume() {
        double suma = 0;
        for (int i = 0; i < liczbaPozycji; i++) {
            // Faktura korzysta z komponentów:
            suma += pozycje[i].obliczWartosc();
        }
        return suma;
    }
}
```

Kompozycja a dziedziczenie

- Kompozycja, podobnie jak dziedziczenie, jest sposobem na ponowne wykorzystanie kodu.
- Zamiast rozszerzać klasy i tworzyć hierarchie, składamy obiekt z innych i delegujemy im pewne zadania.
- Dziedziczenie to relacja "**jest**" (*is-a*), podczas gdy kompozycja to relacja "**ma**" (*has-a*).
- Kompozycja jest dynamiczna i bardziej elastyczna, stąd jest preferowaną techniką (*Composition over inheritance*).

Zamiast **dziedziczenia**:

```
public class Employee {  
    protected String name;  
    protected double salary;  
}  
public class Manager extends Employee {  
    private int teamSize;  
}
```

Możemy użyć **kompozycji**:

```
public class PersonalData {  
    private String name;  
    private double salary;  
}  
  
public class Manager {  
    private PersonalData data;  
    private int teamSize;  
}
```

Temat uzupełniający:

Typy wyliczeniowe i klasy zagnieżdżone

Nie są bezpośrednio związane z kompozycją, ale w praktyce często używane razem z nią.

- typy wyliczeniowe – modelowanie stanów
- klasy wewnętrzne – hermetyzacja

Typy wyliczeniowe (enum)

Typ wyliczeniowy (ang. *enumeration*): to typ danych, który pozwala zdefiniować **zbiór nazwanych stałych**.

Przykład:

```
enum DayOfWeek {  
    MONDAY, TUESDAY, ..., SUNDAY  
}
```

- Poprawia **czytelność i bezpieczeństwo typów** (zamiast „magicznych” liczb/stringów)
- Ułatwia **modelowanie stanów, trybów, kategorii**
- Pomaga w **czytelnym kodzie warunkowym** (`switch` , `if`)
- W Javie/C# można łatwo uzyskać listę wszystkich wartości (`Enum.values()` / `Enum.GetValues()`)
- **Typowe zastosowania**: dni tygodnia, stan zamówienia, kierunki ruchu, rodzaj użytkownika, typ wiadomości/logów,

Często używane jako elementy składowe obiektów (np. status zamówienia), czyli pojawiają się w kompozycji.

enum w C++

```
enum class Status {  
    OK,  
    ERROR,  
    TIMEOUT  
};  
  
Status s = Status::OK;
```

Można porównywać tylko z tym samym typem: `if (s == Status::ERROR)`

enum w Javie

```
public enum Status {  
    OK, ERROR, TIMEOUT;  
  
    public boolean isError() {  
        return this == ERROR || this == TIMEOUT;  
    }  
}
```

- W Javie każdy `enum` to **pełnoprawna klasa** – można dodawać metody, pola, konstruktory.

enum w C#

```
enum Status {  
    OK = 0,  
    Error = 1,  
    Timeout = 2  
}  
  
Status s = Status.OK;
```

- enum w C# to **typ wartościowy** (`value type`), oparty domyślnie na `int` , ale można zmienić typ bazowy:

```
enum Status : byte { OK, Error, Timeout }
```

Klasy wewnętrzne i zagnieżdżone

Nested/inner classes

- Klasa zagnieżdżona to **klasa zadeklarowana wewnątrz innej klasy**.
- W zależności od języka może być:
 - **statyczna** (nie ma dostępu do instancji klasy zewnętrznej),
 - **niestatyczna / wewnętrzna** (może korzystać z danych instancji klasy zewnętrznej).
- C++, C# - tylko klasy statyczne (zagnieżdżone)
- Java - klasy zagnieżdżone lub wewnętrzne - do wyboru

C++ - klasa zagnieżdżona

```
class Zewnetrzna {  
public:  
    class Wewnetrzna {  
    public:  
        void przedstaw() {  
            std::cout << "To jest klasa wewnętrzna w C++\n";  
        }  
    };  
};
```

Użycie:

```
Zewnetrzna::Wewnetrzna obiekt;  
obiekt.przedstaw();
```

- Cel: hermetyzacja.
 - Klasa wewnętrzna ma określony poziom dostępu.
 - Klasa wewnętrzna ma dostęp do prywatnych składowych klasy zewnętrznej (to nie narusza hermetyzacji, gdyż klasa wewnętrzna jest częścią implementacji).

C# – tylko klasy statycznie zagnieżdżone

```
public class Zewnetrzna {  
    public string Imie = "Jan";  
  
    public class Wewnetrzna {  
        public void PrzedstawSie() {  
            Console.WriteLine("To jest klasa wewnętrzna w C#");  
            // Nie można odwołać się do `Imie` bez przekazania referencji  
        }  
    }  
}
```

Użycie:

```
var z = new Zewnetrzna();  
var w = new Zewnetrzna.Wewnetrzna();  
w.PrzedstawSie();
```

Java – klasa statyczna i niestatyczna

```
public class Zewnetrzna {  
    private String dane = "prywatne dane";  
  
    public class Wewnetrzna {  
        public void pokaz() {  
            System.out.println("Dostęp do: " + dane); // działa!  
        }  
    }  
  
    public static class Statyczna {  
        public void pokaz() {  
            System.out.println("To jest statyczna klasa wewnętrzna w Javie");  
        }  
    }  
}
```

Zastosowanie klas wewnętrznych i zagnieżdżonych

- Klasa `Faktura` zawiera wewnętrzną klasę `Pozycja`,
- `Pozycja` reprezentuje jedną linię faktury i jest używana tylko wewnątrz `Faktura`,
- Klasa `Pozycja` jest zagnieżdżona, by ukryć ją przed resztą systemu.

```
public class Faktura
{
    private string numer;
    private Pozycja[] pozycje;
    private int liczbaPozycji;

    public void DodajPozycje(string nazwa, int ilosc, double cena)
    {
        pozycje[liczbaPozycji++] = new Pozycja(nazwa, ilosc, cena);
    }

    // Zagnieżdżona klasa Pozycja – tylko na użytek Faktury
    private class Pozycja
    {
        private string nazwaProduktu;
        private int ilosc;
        private double cenaJednostkowa;

        public Pozycja(string nazwa, int ilosc, double cena) { ... }
    }
}
```

Kolekcje z biblioteki
standardowej

Kolekcje z biblioteki standardowej

- **Kolekcja** to struktura danych, która umożliwia przechowywanie wielu obiektów razem – w sposób uporządkowany, dostępny i efektywny.
 - Ułatwiają organizowanie danych w pamięci.
 - Zapewniają gotowe mechanizmy dodawania, wyszukiwania, usuwania i przeglądania elementów.

Kolekcje są najczęstszym sposobem realizacji kompozycji "jeden obiekt zawiera wiele innych".

- Biblioteki standardowe C++, Javy i C# oferują szereg kolekcji gotowych do użycia.
- Najczęściej spotykane typy kolekcji to:
 - **Listy** – przechowują dane w ustalonej kolejności (`List` , `std::vector`)
 - **Zbiory** – przechowują unikalne elementy (`Set` , `HashSet`)
 - **Mapy** – przechowują pary klucz-wartość (`Map` , `Dictionary` , `std::map`)

Ten fragment ma charakter przeglądowy:

- **niepotrzebne** – uczenie się wszystkich metod na pamięć,
- **potrzebne** – rozmienie co oferuje każdy typ kolekcji i kiedy po nie sięgać:
 - `List` – gdy potrzebujemy kolejności i dostępu po indeksie,
 - `Set` – gdy ważna jest unikalność,
 - `Map` / `Dictionary` – gdy potrzebny jest dostęp po kluczu.

Kolekcje biblioteki standardowej języka C#

C# - List<T>

- Klasa z przestrzeni nazw `System.Collections.Generic`
- Dynamiczna tablica: automatycznie zwiększa rozmiar w miarę dodawania elementów
- Typowana generycznie (`List<int>` , `List<string>` , itd.)
- Szybki dostęp po indeksie ($O(1)$), ale wstawianie/usuwanie w środku - wolniejsze ($O(n)$)

```
var nums = new List<int> { 1, 2, 3, 4 };
```

Na marginesie:

- `var` służy do deklaracji zmiennej o typie wywnioskowanym przez kompilator.

```
var number = 42;
```

- Jest to typ statyczny - nie zmienia to zasad typowania, tylko upraszcza kod.

C# - List<T>

Najważniejsze właściwości i metody

- `Add(item)` - Dodaje element na końcu
- `Insert(index, item)` - Wstawia element w danym miejscu
- `Remove(item)` - Usuwa pierwsze wystąpienie elementu
- `RemoveAt(index)` - Usuwa element na podanym indeksie
- `Contains(item)` - Sprawdza, czy lista zawiera dany element
- `IndexOf(item)` - Zwraca indeks pierwszego wystąpienia elementu
- `Count` - Liczba elementów w liście
- `Clear()` - Czyści listę
- `Sort()` - Sortuje listę
- `Reverse()` - Odwraca kolejność elementów
- `ToArray()` - Zwraca tablicę z elementami listy
- `ForEach(Action<T>)` - Wykonuje akcję na każdym elemencie

C# - List<T>

Przykład użycia

```
using System;
using System.Collections.Generic;

// Tworzenie listy
List<string> fruits = new List<string>();

// Dodawanie elementów
fruits.Add("Apple");
fruits.Add("Banana");
fruits.Add("Cherry");

// Wstawienie elementu na indeks 1
fruits.Insert(1, "Mango");

// Usuwanie elementów
fruits.Remove("Banana"); // po wartości
fruits.RemoveAt(0);      // po indeksie
```

```
// Sprawdzanie
bool hasMango = fruits.Contains("Mango");
int index = fruits.IndexOf("Cherry");

// Iteracja
foreach (string fruit in fruits)
{
    Console.WriteLine(fruit);
}

// Sortowanie i odwracanie
fruits.Sort();
fruits.Reverse();

// Wypisanie elementów
Console.Write(string.Join(", ", fruits));
```

C# - Dictionary<TKey, TValue>

- Typowana kolekcja z przestrzeni nazw `System.Collections.Generic`
- Służy do przechowywania **par klucz-wartość**
- Klucze muszą być **unikalne**
- Operacje `Add`, `Remove`, `TryGetValue`, `ContainsKey`, `[]` - szybkie (średnio $O(1)$)

Wymagania dla kluczy

- Klucz nie może być `null` (dla referencji)
- Typ klucza powinien poprawnie implementować `Equals()` i `GetHashCode()`

Typowe zastosowania

- Liczniki: zliczanie wystąpień słów, liter, obiektów
- Mapowanie: np. ID użytkownika → obiekt użytkownika
- Grupy danych: np. produkty po kategoriach

C# - Dictionary<TKey, TValue>

Najważniejsze metody i właściwości

- `Add(key, value)` - Dodaje parę klucz-wartość
- `dict[key] = value` - Ustawia wartość dla klucza (dodaje lub nadpisuje)
- `TryGetValue(key, out val)` - Pobiera wartość bez wyjątku, jeśli klucz istnieje
- `Remove(key)` - Usuwa wpis z podanym kluczem
- `ContainsKey(key)` - Sprawdza, czy klucz istnieje
- `ContainsValue(value)` - Sprawdza, czy istnieje wartość
- `Count` - Liczba elementów
- `Clear()` - Usuwa wszystkie pary
- `Keys` - Kolekcja wszystkich kluczy
- `Values` - Kolekcja wszystkich wartości
- `foreach (var kvp in dict)` - Iteracja po parach klucz-wartość
- Można zainicjalizować słownik:

```
var capitals = new Dictionary<string, string>
{
    [ "Poland" ] = "Warsaw",
```

C# - Dictionary<TKey, TValue>

Przykład użycia

```
using System;
using System.Collections.Generic;

// Tworzenie słownika
Dictionary<string, int> ages = new();

// Dodawanie danych
ages.Add("Alice", 30);
// alternatywny sposób
ages["Bob"] = 25;

// Sprawdzanie istnienia
if (ages.ContainsKey("Alice"))
{
    Console.WriteLine("Alice ma " +
        ages["Alice"] + " lat.");
}

// Bezpieczne pobieranie wartości
if (ages.TryGetValue("Charlie",
    out int age))
{
    Console.WriteLine("Charlie: " + age);
}
```

```
else
{
    Console.WriteLine("Brak danych.");
}

// Iteracja po słowniku
foreach (var pair in ages)
{
    Console.WriteLine(
        $"{pair.Key} → {pair.Value}");
}

// Usuwanie elementu
ages.Remove("Bob");

// Wyświetlenie liczby elementów
Console.WriteLine("Liczba wpisów: " +
    ages.Count);
```

C# - HashSet<T>

- Kolekcja z przestrzeni nazw `System.Collections.Generic`
- Przechowuje **niewpowtarzalne (unikalne)** elementy
- Działa szybko dzięki wykorzystaniu **tablicy mieszającej (hash table)**
- Szybkie operacje dodawania, usuwania i sprawdzania obecności ($O(1)$ w typowym przypadku)
- Świetny do zliczania, usuwania duplikatów i operacji zbiorowych
- Brak dostępu po indeksie (nieuporządkowane dane)
- Nie zachowuje kolejności dodania.
- **Przykład:** filtrowanie duplikatów

```
var words = new[] { "a", "b", "a", "c" };  
var unique = new HashSet<string>(words);
```

Wymagania

- Typ `T` musi poprawnie implementować `Equals()` i `GetHashCode()`, ponieważ `HashSet` opiera się na haszowaniu.

C# - HashSet<T>

Najważniejsze metody i właściwości

- `Add(item)` - Dodaje element jeśli nie istnieje
- `Contains(item)` - Sprawdza, czy element istnieje
- `Remove(item)` - Usuwa element
- `Count` - Liczba elementów
- `Clear()` - Czyści zbiór
- `SetEquals(otherSet)` - Sprawdza, czy dwa zbiory są równe
- `UnionWith(otherSet)` - Suma zbiorów
- `IntersectWith(otherSet)` - Część wspólna zbiorów
- `ExceptWith(otherSet)` - Różnica zbiorów
- `SymmetricExceptWith(otherSet)` - Elementy różne w obu zbiorach
- `IsSubsetOf(...)` - Czy jest podzbiorem
- `IsSupersetOf(...)` - Czy jest nadzbiorem
- `Overlaps(...)` - Czy ma wspólne elementy

C# - HashSet<T>

Przykład użycia

```
using System;
using System.Collections.Generic;

// Tworzenie HashSet
HashSet<string> fruits = new();

// Dodawanie
fruits.Add("Apple");
fruits.Add("Banana");
fruits.Add("Apple"); // już istnieje

// Sprawdzanie obecności
Console.WriteLine(
    fruits.Contains("Banana")); // true

// Usuwanie
fruits.Remove("Apple");

// Iteracja
foreach (string fruit in fruits)
{
    Console.WriteLine(fruit);
}
```

```
// Operacje zbiorowe
var tropical = new HashSet<string> {
    "Banana",
    "Mango"
};

// suma
fruits.UnionWith(tropical);
// przecięcie
fruits.IntersectWith(tropical);
// różnica
fruits.ExceptWith(new[] { "Mango" });

// Liczba elementów
Console.WriteLine(
    "Liczba owoców: " + fruits.Count);
```

C# – inne kolekcje

- `Queue<T>` – kolejka FIFO (*First In, First Out*).
 - Zawiera m.in. metody `Enqueue(item)`, `Dequeue()`, `Peek()`.
- `Stack<T>` – stos LIFO (*Last In, First Out*).
 - Metody `Push(item)`, `Pop()`, `Peek()`.
- `SortedDictionary<TKey, TValue>`, `SortedList<TKey, TValue>`, `SortedSet<T>` – przechowują elementy w porządku rosnącym (wg klucza lub wartości).
- `OrderedDictionary<TKey, TValue>` – zachowuje kolejność dodawania elementów.
- `PriorityQueue<TElement, TPriority>` – kolejka priorytetowa: elementy są pobierane nie w kolejności dodania, lecz **wg przypisanego priorytetu**.
- `LinkedList<T>` – dwukierunkowa lista powiązana (doubly-linked list).
 - W odróżnieniu od `List<T>`, **szybsza dla wstawek/usuwań w środku listy**, ale wolniejsza w dostępie po indeksie.

Kolekcje biblioteki standardowej języka Java

Java – List<E>

- To **interfejs** z pakietu `java.util`, reprezentujący **kolekcję uporządkowaną**, w której:
 - Elementy mają **indeksy** ($0..n-1$)
 - Dopuszcza **duplikaty**
- Najczęściej używane implementacje to:
 - `ArrayList<E>` – dynamiczna tablica
 - `LinkedList<E>` – lista powiązana
 - `ArrayList<E>` oferuje szybki dostęp po indeksie ($O(1)$), ale wolniejsze wstawianie/usuwanie „w środku” ($O(n)$)

```
import java.util.*;  
  
List<String> list = new ArrayList<>();
```

Java – List<E>

Najważniejsze metody interfejsu

- `add(E element)` – Dodaje element na końcu listy
- `add(int index, E element)` – Wstawia element na podanym indeksie
- `get(int index)` – Zwraca element z danego indeksu
- `set(int index, E element)` – Ustawia nową wartość na podanym indeksie
- `remove(int index)` – Usuwa element na danym indeksie
- `remove(Object obj)` – Usuwa pierwsze wystąpienie obiektu
- `contains(Object obj)` – Sprawdza, czy lista zawiera element
- `indexOf(Object obj)` – Zwraca indeks pierwszego wystąpienia
- `isEmpty()` – Czy lista jest pusta
- `size()` – Liczba elementów
- `clear()` – Czyści listę
- `toArray()` – Zwraca tablicę z elementami
- `subList(from, to)` – Tworzy widok fragmentu listy
- `forEach(...)` – Iteruje po elementach (od Java 8)

Java – List<E>

Przykład użycia

```
List<String> fruits = new ArrayList<>();

fruits.add("Apple");
fruits.add("Banana");
fruits.add("Cherry");

// wstawienie
fruits.add(1, "Mango");

// usunięcie po wartości
fruits.remove("Banana");

// usunięcie po indeksie
fruits.remove(0);
```

```
if (fruits.contains("Mango")) {
    System.out.println(
        "Mango jest na liście");
}

for (String fruit : fruits) {
    System.out.println(fruit);
}

// sortowanie
Collections.sort(fruits);
System.out.println("Po sortowaniu:");
System.out.println(
    String.join(", ", fruits));
```

Java – Map<K, V>

- To **interfejs** reprezentujący **asocjacyjną strukturę danych (słownik/mapa)**:
 - **Klucze (K) są unikalne**, każdemu przypisana jest dokładnie jedna wartość (V).
- Nie dziedziczy po `Collection`, ale jest częścią `java.util`.

Najczęściej używane implementacje:

- `HashMap<K, V>` – Najszybsza, nie gwarantuje kolejności
- `TreeMap<K, V>` – Klucze posortowane rosnąco (`Comparable` lub `Comparator`)
- `LinkedHashMap<K, V>` – Zachowuje **kolejność dodania**

Uwaga

- **Klucze muszą być unikalne** – nowy wpis z tym samym kluczem nadpisze poprzedni.
- Klucze powinny poprawnie implementować `equals()` i `hashCode()` (dla `HashMap`).

Java – Map<K, V>

Najważniejsze metody interfejsu Map<K, V>

- `put(K key, V value)` – Dodaje lub aktualizuje wartość
 - `putIfAbsent` – Wstawienie tylko jeśli nie istnieje
 - `computeIfPresent` – Modyfikacja tylko jeśli istnieje
- `get(Object key)` – Zwraca wartość dla klucza
 - `getOrDefault` – Bezpieczne pobieranie z domyślną wartością
- `remove(Object key)` – Usuwa wpis
- `containsKey(Object key)` – Sprawdza, czy klucz istnieje
- `containsValue(Object val)` – Sprawdza, czy wartość istnieje
- `size()` – Liczba par
- `isEmpty()` – Czy mapa jest pusta
- `clear()` – Czyści mapę
- `keySet()` – Zbiór kluczy (`Set<K>`)
- `values()` – Kolekcja wartości (`Collection<V>`)
- `entrySet()` – Zbiór wpisów (`Set<Map.Entry<K, V>>`)
 - Iteracja po `entrySet()` jest najwydajniejsza, jeśli potrzebujemy jednocześnie klucza i wartości.

Java – Map<K, V>

Przykład użycia

```
Map<String, Integer> ages =
    new HashMap<>();

// Dodawanie
ages.put("Alice", 30);
ages.put("Bob", 25);
ages.put("Charlie", 28);

// Nadpisanie wartości
ages.put("Alice", 31);

// Odczyt
System.out.println("Wiek Alicji: " +
    ages.get("Alice"));

// Sprawdzanie istnienia
if (ages.containsKey("Bob")) {
    System.out.println(
        "Bob jest w mapie.");
}
```

```
// Iteracja po parach
for (Map.Entry<String, Integer> entry :
    ages.entrySet()) {
    System.out.println(
        entry.getKey() +
        " → " +
        entry.getValue());
}

// Usuwanie
ages.remove("Charlie");
System.out.println(
    "Rozmiar mapy: " + ages.size());
```

Java – Set<E>

- To interfejs z pakietu `java.util`, który reprezentuje **zbiór unikalnych elementów – bez duplikatów**.
- W przeciwieństwie do `List`, elementy w `Set` **nie mają indeksów**.
- Działa na zasadzie matematycznego zbioru.

Najczęściej używane implementacje:

- `HashSet<E>` – Najszybszy, brak gwarancji kolejności
- `LinkedHashSet<E>` – Zachowuje **kolejność dodania**
- `TreeSet<E>` – Elementy są **sortowane rosnąco** (wg `Comparable` lub `Comparator`)

Uwaga:

- Elementy powinny poprawnie implementować `equals()` i `hashCode()` (szczególnie ważne w `HashSet`).

Java – Set<E>

Najważniejsze metody interfejsu

- `add(E element)` – Dodaje element (jeśli nie istnieje)
- `remove(Object element)` – Usuwa element (jeśli istnieje)
- `contains(Object element)` – Sprawdza, czy element istnieje
- `size()` – Liczba elementów
- `isEmpty()` – Czy zbiór jest pusty
- `clear()` – Czyści zbiór
- `iterator()` – Zwraca iterator
- `forEach(...)` – Iteruje po elementach (od Java 8)

Java – Set<E>

Przykład użycia

```
Set<String> names = new HashSet<>();

names.add("Alice");
names.add("Bob");
names.add("Charlie");
names.add("Alice"); // zignorowane – już istnieje

System.out.println("Rozmiar: " + names.size()); // 3

// Sprawdzanie
if (names.contains("Bob")) {
    System.out.println("Bob jest w zbiorze");
}

// Iteracja
for (String name : names) {
    System.out.println(name);
}

// Usunięcie
names.remove("Charlie");
```

Java – Queue<E>

kolejka (FIFO)

- Interfejs z `java.util`, reprezentujący **kolejkę FIFO (First In, First Out)**.
- Główne zastosowanie: **przetwarzanie danych w kolejności ich przyścia**.
- Rozszerza `Collection<E>`.

Najczęstsze implementacje

- `LinkedList<E>` – Pełna implementacja kolejki i listy
- `ArrayDeque<E>` – Szybka implementacja kolejki i stosu
- `PriorityQueue<E>` – Kolejka oparta na **priorytetach** (elementy sortowane)

Java – Queue<E>

Najważniejsze metody

- `add(e)` - Dodaje element (wyjątek, jeśli brak miejsca)
- `offer(e)` - Dodaje element (zwraca `false` , jeśli nie dodano)
- `remove()` - Usuwa i zwraca pierwszy element (wyjątek, jeśli pusto)
- `poll()` - Usuwa i zwraca pierwszy element (`null` , jeśli pusto)
- `element()` - Zwraca pierwszy element bez usuwania (wyjątek)
- `peek()` - Zwraca pierwszy element bez usuwania (`null` , jeśli pusto)

Java – Deque<E>

Kolejka dwukierunkowa (FIFO + LIFO)

- Deque (Double-Ended Queue) to **kolekcja, która pozwala dodawać i usuwać elementy na obu końcach**.
- Rozszerza Queue<E> , dodaje metody dla operacji LIFO (jak Stack).

Najczęstsze implementacje

- ArrayDeque<E> – Najszybsza implementacja Deque
- LinkedList<E> – Też implementuje Deque

Java – Deque<E>

Najważniejsze metody

- `addFirst(e)` – Dodaje element na początek
- `addLast(e)` – Dodaje element na koniec
- `removeFirst()` – Usuwa i zwraca element z początku
- `removeLast()` – Usuwa i zwraca element z końca
- `peekFirst()` – Podgląd początku
- `peekLast()` – Podgląd końca
- `offerFirst(e)` – Dodaje na początek (bez wyjątku)
- `offerLast(e)` – Dodaje na koniec (bez wyjątku)
- `pollFirst()` / `pollLast()` – Usuwa i zwraca (lub `null`)

Java – Queue<E>, Deque<E>

Przykład użycia

■ Queue<E>

```
Queue<String> tasks = new LinkedList<>();

tasks.offer("Mail");
tasks.offer("Call");
tasks.offer("Code");

System.out.println("Next: " + tasks.peek());    // Mail
System.out.println("Handling: " + tasks.poll()); // Mail
System.out.println("Next: " + tasks.peek());    // Call
```

■ Deque<E>

```
Deque<String> deque = new ArrayDeque<>();

deque.addFirst("A");    // -> A
deque.addLast("B");     // -> A, B
deque.addLast("C");     // -> A, B, C

System.out.println(deque.removeFirst()); // A
System.out.println(deque.removeLast());  // C
```

Kolekcje biblioteki standardowej języka C++

Najważniejsze kolekcje STL

STL – *Standard Template Library*

- `std::vector<T>` – Dynamiczna tablica (najczęściej używana)
- `std::array<T, N>` – Tablica statyczna z rozmiarami znanymi w czasie kompilacji
- `std::list<T>` – Dwukierunkowa lista powiązana
- `std::deque<T>` – Dwustronna kolejka/tablica
- `std::stack<T>` – Stos (adaptor)
- `std::queue<T>` – Kolejka FIFO (adaptor)
- `std::priority_queue<T>` – Kolejka priorytetowa (adaptor)
- `std::set<T>` – Zbiór uporządkowany bez duplikatów
- `std::map<K,V>` – Słownik uporządkowany
- `std::unordered_set<T>` – Zbiór nieuporządkowany (hash)
- `std::unordered_map<K,V>` – Mapa nieuporządkowana (hash)

Sekwencyjne kontenery (vector, list, deque)

- `std::vector<T>` - działa jak dynamiczna tablica
 - Szybki dostęp po indeksie $O(1)$
 - Automatycznie rozszerzanie rozmiaru
 - Najczęściej używany kontener

```
std::vector<int> v = {1, 2, 3};  
v.push_back(4); // dodaje na końcu  
v[0] = 10;  
v.at(1);        // bezpieczny dostęp  
v.size();       // liczba elementów  
v.empty();      // czy pusty  
v.clear();      // usuwa wszystkie elementy
```

Asocjacyjne kontenery (set , map)

- `std::set<T>` - przechowuje unikalne elementy w porządku rosnącym (domyślnie wg `<`)
 - Szybkie operacje: `insert` , `erase` , `find` - $O(\log n)$

```
std::set<int> s;  
s.insert(10);  
s.erase(10);  
s.count(10);    // 0 lub 1  
s.find(10);     // iterator do elementu
```

- `std::map<K,V>` - przechowuje unikalne klucze, powiązane z wartościami
 - Klucze automatycznie sortowane rosnąco (drzewo binarne)
 - Szybkie operacje: `insert` , `erase` , `find` , `operator[]` - $O(\log n)$
 - `map[k]` tworzy nowy wpis, jeśli klucz nie istnieje

```
std::map<std::string, int> m;  
m["apple"] = 2;  
m.at("apple");    // bezpieczny dostęp  
m.erase("apple");  
m.count("apple");
```

Adaptory kontenerów (stos, kolejka, kolejka priorytetowa)

- `std::stack<T>`

```
std::stack<int> st;  
st.push(1);  
st.top();      // podgląd  
st.pop();
```

- `std::queue<T>`

```
std::queue<int> q;  
q.push(1);  
q.front();     // pierwszy element  
q.pop();
```

- `std::priority_queue<T>`

```
std::priority_queue<int> pq;  
pq.push(5);  
pq.push(2);  
pq.top();      // największy element  
pq.pop();
```